

CA1 Project 1: Build a Real Agent

Maximum Marks: 30 | Last date: 26 August 2026 | Solo or group of up to 3

What you will build

You will build **one AI agent** that helps with an everyday task. Pick **one topic** from the list of 30 below. Every project has the same simple shape:

- **One agent** with a plan-act loop (it takes more than one step).
- **At least two tools** the agent calls (small Python functions are fine).
- **Memory** so it remembers earlier turns in the same conversation.

You may use **any lane from the course** (free GitHub Models or Microsoft Foundry, your choice). Marks do not depend on the lane.

The one rule: an agent, not a chatbot

The simplest way to think about it: **a chatbot answers, an agent acts**. A chatbot is a vending machine, one question in, one answer out. An agent is an intern, you give it a goal and it decides which tools to use, does several steps, remembers what it found, and comes back with a result.

Your project **must** show all three of these, or it does not meet the course outcomes:

- It **calls tools** (at least two), not just generates text.
- It takes **more than one step**, deciding what to do next from tool results.
- It **remembers** earlier turns and uses that memory later.

A project that only does "user asks, model replies" will not pass, however polished it looks.

Solo or group of three

You may work **alone or in a group of up to three**. Same topics for both. A solo project builds the core shape (two tools plus memory). A group of three also builds the **Group of 3 add-on** listed under the topic, so the workload is fair.

What to submit

Submit a **single zip** (or a GitHub link) containing:

- Your **code** (the agent and its tools).
- A **short README**: name the two tools, say what the memory does, and describe one honest failure you hit and how you handled it. Three short paragraphs, no essay.
- A **notebook demo** that runs the agent on 2 to 3 example goals and shows the multi-step trace (the tool calls and steps). This is your proof it is an agent, not a chatbot.

Group submissions: add a one-line **who-did-what** note in the README.

How it is marked (30)

Criterion	What we look for	Marks
Implementation	The agent itself, working. A real plan-act loop that takes more than one step and uses tool results to decide the next step; at least two working tools the agent actually calls (not just text); and memory that remembers earlier turns and is used in a later answer.	10
Presentation	Your demo and write-up. A notebook that runs on 2 to 3 example goals and	10

Criterion	What we look for	Marks
	shows the multi-step trace (the tool calls and steps), plus a short, clear README naming the two tools, what memory does, and one honest failure. If the notebook does not run, implementation cannot be verified.	
Viva	You explaining your own agent. Be ready to point to where the plan-act loop decides the next step, walk through a tool call, and show where memory is read back. The viva confirms the work is yours and that you understand it.	10

Note: the **running notebook** is what proves the agent is real. If it does not run, the implementation marks cannot be verified. These three headings (Implementation, Presentation, Viva) are the official rubric for Project 1; the detail under each is what earns the marks within it.

Choose one of these 30 topics

Topics are grouped by domain: finance, travel, health, and study. All are the same difficulty. Pick the one you find most interesting.

Finance

T1. Personal Budget Assistant [Finance]

Goal: Help a student track a monthly budget and answer money questions.

Two tools: `add_expense(item, amount)`, `get_summary(category)`

Memory: Remembers every expense in the session so category totals stay correct.

Must be agentic: When asked "can I afford a 2000 trip?", it checks the balance, subtracts, and decides, not just chats.

Group of 3 add-on: Add `set_savings_goal` and a warning when a category is overspent.

T2. Bill Splitter Agent [Finance]

Goal: Split shared costs among friends and work out who owes whom.

Two tools: `add_share(person, amount)`, `settle_up()`

Memory: Remembers who paid what across the whole conversation.

Must be agentic: It computes the net balances and decides the simplest set of repayments.

Group of 3 add-on: Handle uneven splits and add a per-person summary.

T3. Savings Goal Tracker [Finance]

Goal: Help a user save towards a target amount by a date.

Two tools: `set_goal(amount, months)`, `log_saving(amount)`

Memory: Remembers progress so far and the goal.

Must be agentic: It decides whether the user is on track and how much per month is still needed.

Group of 3 add-on: Add a catch-up plan when the user falls behind.

T4. Subscription Manager [Finance]

Goal: Track monthly subscriptions and control their total cost.

Two tools: `add_subscription(name, cost)`, `get_monthly_total()`

Memory: Remembers all active subscriptions.

Must be agentic: When over a budget, it decides which subscriptions to suggest cancelling to get back under.

Group of 3 add-on: Add renewal-date reminders and a yearly-cost view.

T5. Trip Currency Budgeter [Finance]

Goal: Keep a foreign-trip budget under control in the home currency.

Two tools: `convert_currency(amount, from, to)`, `add_expense(item, amount)`

Memory: Remembers spending so far in home currency.

Must be agentic: It converts each expense and decides if the running total is still within budget.

Group of 3 add-on: Add a daily allowance and warn when a day is overspent.

T6. EMI and Loan Advisor [Finance]

Goal: Help a user compare simple loan options.

Two tools: `compute_emi(amount, rate, months)`, `compare_options()`

Memory: Remembers the user's stated monthly income and options seen.

Must be agentic: It decides which option is affordable given income, and explains why.

Group of 3 add-on: Add total-interest comparison across options.

T7. Expense Category Sorter [Finance]

Goal: Sort spending into categories and find where money leaks.

Two tools: `classify_expense(item)`, `get_category_total(category)`

Memory: Remembers all classified expenses.

Must be agentic: It decides the biggest overspending category and suggests one cut.

Group of 3 add-on: Add a month-on-month change once two months are logged.

T8. Simple Invoice Assistant *[Finance]*

Goal: Build a small invoice with tax and discounts.

Two tools: `add_item(name, price, qty)`, `compute_total(tax_percent)`

Memory: Remembers all line items added.

Must be agentic: It decides whether a discount threshold is met and applies it.

Group of 3 add-on: Add multiple tax slabs and a formatted invoice summary.

Travel

T9. Weekend Trip Planner *[Travel]*

Goal: Plan a short trip that fits a set number of days.

Two tools: `find_activities(city)`, `add_to_itinerary(activity)`

Memory: Remembers the itinerary built so far.

Must be agentic: It decides whether a new activity fits the remaining days before adding it.

Group of 3 add-on: Add rough timing per activity and flag an overpacked day.

T10. Flight Option Finder *[Travel]*

Goal: Find and pick a flight within budget.

Two tools: `search_flights(from, to)`, `compare_price(options)`

Memory: Remembers the user's budget and preferences.

Must be agentic: It decides the cheapest option under budget, or reports none fits.

Group of 3 add-on: Add a non-stop-preferred rule and a second-best fallback.

T11. Smart Packing List Agent *[Travel]*

Goal: Build a packing list that matches the trip and weather.

Two tools: `check_weather(city)`, `add_item(item)`

Memory: Remembers the trip type and items added.

Must be agentic: It decides what to pack based on the weather it looked up, not a fixed list.

Group of 3 add-on: Add a "forgot something?" check against trip type.

T12. Hotel Shortlister *[Travel]*

Goal: Shortlist hotels that fit a budget and must-haves.

Two tools: `search_hotels(city)`, `filter_by_budget(max_price)`

Memory: Remembers the user's must-haves (wifi, breakfast).

Must be agentic: It decides the best fit and explains the trade-off it made.

Group of 3 add-on: Add a ranking across two or three shortlisted hotels.

T13. Local Food Recommender *[Travel]*

Goal: Recommend places to eat that match diet and taste.

Two tools: `find_restaurants(city)`, `filter_by_cuisine(type)`

Memory: Remembers dietary needs stated earlier.

Must be agentic: It decides suitable options and rules out ones that break the diet.

Group of 3 add-on: Add a distance-aware pick when a location is given.

T14. Multi-Stop Route Planner *[Travel]*

Goal: Order a set of stops into an efficient route.

Two tools: `get_distance(a, b)`, `order_stops(list)`

Memory: Remembers stops already visited.

Must be agentic: It decides a sensible visiting order rather than the input order.

Group of 3 add-on: Add a return-to-start option and total-distance report.

T15. Day-Trip Scheduler *[Travel]*

Goal: Fit attractions into a single day by opening hours.

Two tools: `get_open_hours(place)`, `build_schedule(places)`

Memory: Remembers user preferences and chosen places.

Must be agentic: It decides an order that respects opening hours and skips what will not fit.

Group of 3 add-on: Add travel-time gaps between stops.

Health

T16. Water Intake Coach *[Health]*

Goal: Help a user reach a daily water goal.

Two tools: `log_water(ml)`, `get_progress()`

Memory: Remembers the day's intake so far.

Must be agentic: It decides whether the goal is met and how much is left, and nudges accordingly.

Group of 3 add-on: Add a weekly average and a gentle streak.

T17. Meal Calorie Helper *[Health]*

Goal: Track a day's meals against a calorie budget (informational only).

Two tools: `lookup_calories(food)`, `add_meal(food)`

Memory: Remembers meals added today.

Must be agentic: It decides the remaining budget and whether a planned snack fits.

Group of 3 add-on: Add a simple macro split once foods are logged.

T18. Workout Planner *[Health]*

Goal: Plan and track simple workouts week to week.

Two tools: `suggest_exercise(goal)`, `log_session(exercise)`

Memory: Remembers past sessions.

Must be agentic: It decides the next session based on what was done recently, not a fixed plan.

Group of 3 add-on: Add light progression once a few sessions exist.

T19. Medication Reminder Agent *[Health]*

Goal: Track medicines and their timing (reminders only, not medical advice).

Two tools: `add_medication(name, times)`, `check_schedule(now)`

Memory: Remembers all medicines and their schedule.

Must be agentic: It decides the next dose due and flags anything missed.

Group of 3 add-on: Add a daily checklist view.

T20. Sleep Tracker Assistant *[Health]*

Goal: Track sleep and reflect simple patterns back.

Two tools: `log_sleep(hours)`, `get_average()`

Memory: Remembers each night's hours.

Must be agentic: It decides whether the recent average is below a healthy range and suggests one change.

Group of 3 add-on: Add a best-and-worst-night summary for the week.

T21. Step Goal Coach *[Health]*

Goal: Help a user hit a daily step target.

Two tools: `log_steps(count)`, `get_progress()`

Memory: Remembers steps logged today.

Must be agentic: It decides remaining steps and gives a specific, encouraging nudge.

Group of 3 add-on: Add a weekly total and a small milestone.

T22. Habit Streak Tracker *[Health]*

Goal: Keep users consistent on a chosen habit.

Two tools: log_habit(name), get_streak(name)

Memory: Remembers streaks across habits.

Must be agentic: It decides an encouraging message based on the current streak length.

Group of 3 add-on: Add multiple habits and a "most consistent" summary.

T23. Symptom Information Helper *[Health]*

Goal: Give general information and a when-to-see-a-professional pointer. This is NOT a diagnosis tool, and every answer must say so.

Two tools: lookup_info(symptom), suggest_next_step(symptom)

Memory: Remembers symptoms mentioned in the session.

Must be agentic: It decides a general next step (rest, hydrate, or see a professional) and always advises consulting a doctor for anything serious.

Group of 3 add-on: Add a clear, prominent disclaimer and an "urgent signs, seek care now" list.

Study

T24. Study Planner Agent *[Study]*

Goal: Plan study blocks around real deadlines.

Two tools: add_task(name, due), build_schedule()

Memory: Remembers all tasks and deadlines.

Must be agentic: It decides an order and fits study blocks before each due date.

Group of 3 add-on: Add a re-plan when a new urgent task arrives.

T25. Assignment Deadline Tracker *[Study]*

Goal: Keep track of what is due and what is urgent.

Two tools: add_assignment(name, due), get_upcoming()

Memory: Remembers all assignments.

Must be agentic: It decides which items are urgent and orders them by priority.

Group of 3 add-on: Add a "days left" flag and an overdue list.

T26. Flashcard Quiz Agent *[Study]*

Goal: Quiz a user and focus on their weak spots.

Two tools: add_card(q, a), quiz_me()

Memory: Remembers which cards were answered wrong.

Must be agentic: It decides to re-quiz the weak cards more often, not ask at random.

Group of 3 add-on: Add a score trend across quiz rounds.

T27. Reading List Manager *[Study]*

Goal: Manage a reading list and track progress.

Two tools: add_book(title, pages), track_progress(title, page)

Memory: Remembers progress per book.

Must be agentic: It decides what to read next based on pace and what is unfinished.

Group of 3 add-on: Add a finish-by estimate from recent pace.

T28. GPA Advisor *[Study]*

Goal: Compute GPA and advise on what is needed next.

Two tools: add_grade(course, grade), compute_gpa()

Memory: Remembers all grades entered.

Must be agentic: It decides what grades are needed next term to reach a target GPA.

Group of 3 add-on: Add a best-and-worst-case projection.

T29. Exam Countdown Planner *[Study]*

Goal: Spread exam topics across the days that remain.

Two tools: `set_exam(date)`, `allocate_topics(topics)`

Memory: Remembers the exam date and topics assigned.

Must be agentic: It decides a day-by-day topic plan based on days left.

Group of 3 add-on: Add a catch-up shuffle when a day is missed.

T30. Pomodoro Study Coach *[Study]*

Goal: Run focused study sessions and track focus time.

Two tools: `start_session(minutes)`, `log_session()`

Memory: Remembers completed focus sessions.

Must be agentic: It decides when to suggest a break or another session based on sessions done.

Group of 3 add-on: Add a daily focus-time total and a small goal.

Want a topic that is not listed? Email me a one-line goal and your two tools before you start, and I will approve it if it fits the one rule above.

Health topics are for tracking and general information only. They are not medical tools and must not give diagnoses. Topic 23 in particular must always tell the user to consult a professional.